
# PROGRAMMING PYXEL GAME

**Dual Bachelor in Data Science and Engineering and Telecommunications Engineering**

**Alejandro Barroso - 100499081**
**Juan José Rosales - 100499176**
**Group 196**

**Abstract**

**This project tries to recreate in the most faithful way the arcade game "1942" using object oriented programming with Python. To do it, we used a package called pyxel.**

# Index

# 1. Classes Design

For the execution of this game, we used object oriented programming, where each of the objects that appear in the game belong to a certain class. Besides, we applied the inheritance in some of the classes we had.

### Board

The class Board can be considered as our main class, since it is the class that makes the game work. Inside this class we introduced the three different stages of our game, the *start screen*, which waits till the player presses "Enter" to start the next stage. The *game screen*, which is the most important one, since it creates all the different objects of the game and keeps track of the evolution of the player until it loses all his lives, moving to the *game over screen*. This last stage stops the game and restores all the values to be able to start the game again.

### Player

This class represents the user in the game, updating the different movements done by him in the keyboard and the loop stage, which makes the player untouchable by the enemies. This class changes its sprites to simulate the helix move and the loop done when the user presses "z".

### Enemy

This is the first mother class. In it we create the principal attributes that are common to all the enemies, such as the points they give when they die or the lives they have.

### Regular (Enemy).

This child class represents the most basic enemy, with its special ability (move back randomly).

### Red (Enemy).

This subclass collects the second enemy type, which are the ones that move around the screen doing circles.

### Bombardier (Enemy).

The bombardier is an enemy that has more lives, is bigger and shoots double. Its movement is in zigzag.

### SuperBombardier (Enemy).

The last enemy is the strongest one, and appears at the bottom of the screen moving upwards and keeps shooting until it gets killed.

### Shot

This is another mother class that collects the **EnemyShot** and **PlayerShot**.

### Blast

The Blast class creates an explosion when a life is lost by an enemy or the player.

### Bonus

This extra class is the mother of 4 child classes, one for each Bonus type: **BonusShot**, which gives the player 2 extra shots; **BonusSpeed**, which increases the Player's speed; **BonusLive,** which gives an extra live to the player if he has lost any of them; and **BonusLoop**, which restores a used loop.

## 2. Most relevant fields and methods

On the development of the project, we have introduced many different methods to achieve several functionalities of the game. However, we can agree that the most important ones were the *update* and *draw* methods. The first one is the one that is constantly running, so we can say that is the method that maintains the game alive. The draw one is the method that represents the sprites of each object on screen, being the place where all the animations are made, such as the movement of the player's helix.

Besides, the most relevant fields for us are the lists ones, where we store every single shot, enemy, blast and bonus. However, they would have been useless if we hadn't created some special methods that move along those lists. This methods are:

- *update_list*, which updates constantly each of the elements of the list.
- *draw_list*, which represents on screen the objects of the list.
- *cleanup_list*, which removes from the list the elements whose *is_alive* attribute is False.

Furthermore, we need to highlight the importance of the collision methods that detect the contact between sprites, subtracting a live from the objects involved in the collision. This methods are:

- *collision_shot_enemy,* between each of the player's shots and each of the enemies.
- *collision_shot_player,* between each of the enemies' shots and the player.
- *collision_player_enemy,* between the player and all the enemies on screen.

Finally, we wanted to talk about the *red_killed_enemies* field, which is the counter that eventually will create a random bonus. This attribute is restored back to 0 when a new red enemies wave is generated, and if all five red enemies of the wave get killed by the player, a method called *bonus_creation* gets run.

# 3. Most relevant algorithms

In this project we have created some sets of rules to compute important parts of the game. For example one can be the code necessary to change between the start screen, the game screen and the game over screen. In this case we have defined a variable scene, this variable will change between 0 and 3, depending on the value of the variable and using an if-else statement we can control which part of the game we want to run.

Also, in the class Blast to create the explosion of a plane when it collides with something we use an algorithm. The performance of it is based on two main attributes: radius and is_alive. In this case we display the first 30 frames of the explosion 4 orange and red circles that increase their radius each frame. After the 30 frames this circles disappear so we can see a series of gray circles that simulate the trail of smoke from the explosion.

One of the most important algorithms in our game is the way in which we decided to control the arrays. In this case we have 3 arrays that store the shots, the enemies, the blasts and the bonus. All of them include objects: Enemy,Blast,Shots or Bonus and all these objects have the methods update and draw. To display the game we need each frame to run these methods to obtain a smooth movement of the sprites, so to do it we created 3 methods that are update_list(), cleanup_list() and draw_list(). These methods respectively, update each object in the list, delete objects that are not being used and draw the sprites of each object.

As the last important algorithm we would like to remark on the most complex algorithm of our code. We decided to create some bonus when the player eliminates the whole group of Red enemies. To achieve this task we add a new attribute, RedKilledEnemies, to the player. With this attribute we count the number of Red enemies that the player has killed, each time that a group of enemies appears the attribute is changed to 0 . If it reaches five a bonus will be created in the position of the last enemy killed. These bonuses have their own subclass and are randomly chosen.

# 4. Work performed

In this pyxel project we have tried to recreate as faithfully as possible the 1942 game in which the player has to eliminate a series of different enemies. The game that we have created is based on the idea of a plane that the player can control and with which he can shoot. Throughout the game different enemies will appear that he will have to eliminate to acquire a better score. If the player is hit by enemy bullets or crashes into an enemy, he will lose a life. When the player loses 3 lives, the game is over and the player will have the option to start over.

To program our game we tried to follow the sprints but it was difficult since at the beginning we did not have clear ideas about what we wanted or how to do a correct development of the code. Also, we

didn't know how to use the pyxel library. After installing it and doing a few tries, we started to create the main code that has all the properties related to the screen size, which can be any size, since we have used pyxel.width and pixel.height for all the code to adapt to screen size changes. Also, this code is what will call our main Board class.

The next step was to prepare the class board to update all the necessary information and draw all the necessary sprites on the screen for each frame. We decided to create different methods to update and draw the different scenarios of the game.

Next, we create a class for the player that has its own update and draw methods that will be called to draw the player plane. The updated method allowed us to easily create the plane's motion by adding 1 to the plane's x or y position every frame and changing the plane's sprite every 4 frames. Also, this is where we add the functionality that the player has 3 lives and if they lose all of them, they will die and the game will be over.

In a similar way we develop the enemies, but in this case it is a mother class and 4 subclasses that have their own methods of shooting, moving... in different ways. In summary, there are four types: Regular, with 1 life and straight movement; Red, with 1 life and circle movement; Bombardier with 12 lives, and zigzag movement; and Superbombardier, which appears at the bottom of the screen with 20 lives. The most complex part was to create the circular and zigzag movements that are done with the pyxel methods .sin() and .cos().

Later, we created a class called Shots for all the shots in the game. It has different methods to distinguish enemy shots from player shots. The shots are small sprites that move around the screen and when they collide they disappear and take a life. Related to shots, we created a class called Blasts to generate an explosion when shots or the player collide with an enemy. This explosion is generated by pyxel's .circ() and .circb() methods. They grow with each frame and are visible for 40 frames.

To unite all these classes and display them in the correct frames, we use the class board. In this class we add different methods to create the enemies using the enemies class, to create the shots using the shots class or to calculate the collisions between different objects.

At the end, we added the possibility to make a loop with the plane to avoid the enemies or their shots. To do this, we updated the player's draw method to check if it's looping and change the sprites that are displayed. To avoid getting killed during the loop, we changed the method that kills the player to check if the player is in the loop, and if so, it has no effect.

As extras we have included 4 bonuses that are special features for the plane, such as extra life, faster movement, extra loop and double shot. These bonuses appear on the screen when the player kills all 5 red enemies in a group. To add this functionality we add a new class to our code called bonus, this class has a subclass for each type of bonus, thanks to these classes the game can update the position of the bonuses and the sprite that should be drawn for each type of bonus.

# 5. Conclusion

To sum up, for the creation of this game we used object oriented programming, where we grouped all the functions related to an entity in the same class, having for example a Player class with the functionalities and characteristics of the plane controlled by the user.

This way of programming was a challenge for us, since it was a big step in understanding the way a program works. However, following the steps given in the final project's guide helped us to be ordered in the progression of the game and achieve the objectives of this project.

Besides, during the realization of this project we had to face several difficulties that we needed to solve by ourselves, since the information available online was almost non-existent. However, these drawbacks helped us to understand object oriented programming in a different way, much more practical instead of theoretical.

# 6. Personal comments

This project is one of the most complex we have done and therefore we have found a series of complexities, but with patience and using the notes we had, we were able to solve them. First of all, it is a really free project because we don't have too many instructions to follow and we haven't seen any example or similar code before.

At the beginning it was hard to create a structure for the classes and for the game as we did not know pyxel and the use of object oriented programming was new for us. For this reason, we think that the structure of our game could be better if we had known everything that we have learned by making the game at the beginning.

Another tricky thing that we did was to create the movement of some enemies, precisely the circling and zigzag movement of enemies. In the beginning, we thought that we could replicate the circle movement by an octagon, but it was not good at all. We discovered that pyxel has some useful methods for this purpose like .sin() and .cos(). So we decided to do the movement with these methods. When we started to code this part, the movement of the enemies was random because they didn't start the circle at the correct position, but finally, with the help of Calculus I and some hours we changed some parameters and got really good results.

The hardest part was to distinguish which attributes and methods should be private and read only. We did not know exactly what attributes needed to be private or public and the same for methods. We tried to understand the characteristics of each one by reading the notes and some websites, but it was really abstract and we tried to do our best when setting them.